实验 8　进程通信——共享存储区和信号量

一、实验目的

（1）掌握共享存储和信号量通信原理

（2）基于 openEuler 系统完成共享存储区和信号量通信实验

二、背景知识

2.1 共享存储区机制的概念

共享存储区（Share Memory）是 openEuler 系统中通信速度最高的一种通信机制。该机制可使若干进程共享主存中的某一个区域，且使该区域出现（映射）在多个进程的虚地址空间中。另一方面，一个进程的虚地址空间中又可连接多个共享存储区，每个共享存储区都有自己的名字。当进程间欲利用共享存储区进行通信时，必须先在主存中建立一共享存储区，然后将它附接到自己的虚地址空间上。此后，进程对该区的访问操作，与对其虚地址空间的其它部分的操作完全相同。进程之间便可通过对共享存储区中数据的读、写来进行直接通信。图示列出二个进程通过共享一个共享存储区来进行通信的例子。其中，进程 A 将建立的共享存储区附接到自己的 AA’区域，进程 B 将它附接到自己的 BB’区域。

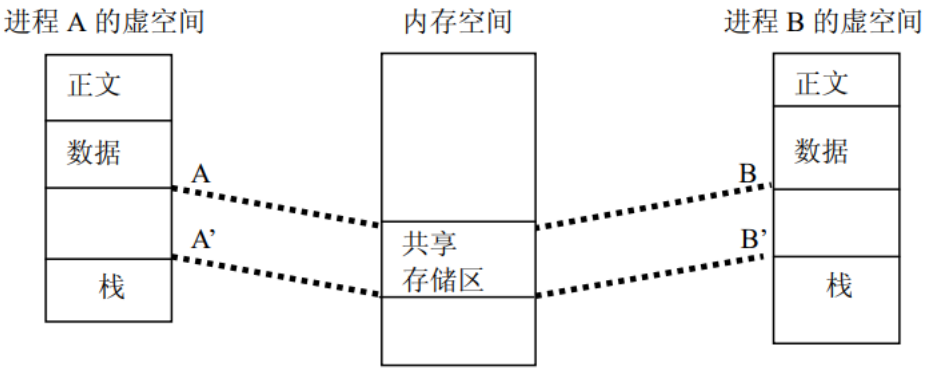


图 1　共享存储区示意图

应当指出，共享存储区机制只为进程提供了用于实现通信的共享存储区和对共享存储区进行操作的手段，然而并未提供对该区进行互斥访问及进程同步的措施。因而当用户需 要使用该机制时，必须自己设置同步和互斥措施才能保证实现正确的通信。

2.2 信号量机制的概念

信号量（semaphore）与其它进程通信方式不同，它主要提供对进程间共享资源访问的控制机制。进程可以根据它判定是否能够访问某共享资源，同时，进程也可以修改它的值。除了用于访问控制外，还可用于进程同步。openEuler 支持的是系统 V 信号量机制，是计数信号量集。

相关数据结构在/usr/include/linux/sem.h 中有定义，图示如下：

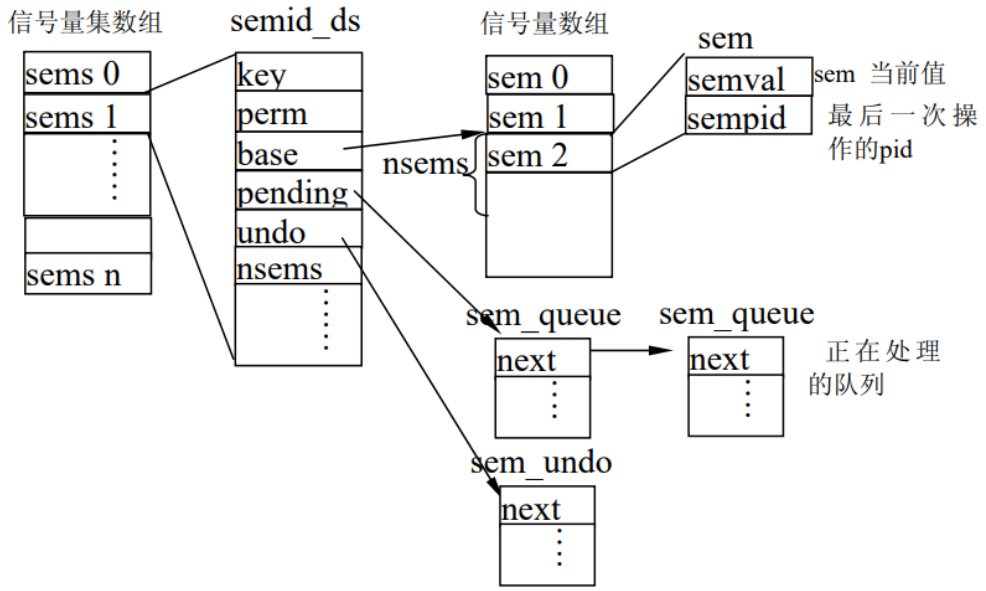


图 2 信号量示意图

三、实验步骤

3.1 共享存储区模拟

共享存储区机制相关系统调用与接口函数：

使用与共享存储区机制有关的系统调用，都需包含以下头文件：

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/shm.h>
```

使用与共享存储区机制有关的接口函数说明如下：

（1）shmget()

创建、获得一个共享存储区。

系统调用格式：

shmid=shmget(key,size,flag)

参数定义：

int shmget(key,size,flag);

key_t key;

int size,flag;

其中，key 是共享存储区的名字；size 是其大小（以字节计）；flag 是用户设置的标志， 如 IPC_CREAT。IPC_CREAT 表示若系统中尚无指名的共享存储区，则由核心建立一个共享存储区；若系统中已有共享存储区，便忽略 IPC_CREAT。

附：

操作允许权	八进制数
用户可读	00400
用户可写	00200
小组可读	00040
小组可写	00020
其它可读	00004
其它可写	00002

控制命令	值
IPC_CREAT	0001000
IPC_EXCL	0002000

例：

shmid=shmget(key, size, IPC_CREAT|0640)

创建一个关键字为 key，长度为 size 的共享存储区，该共享存储区允许用户自己进行读写访问，只允许同组用户进行读访问

（2）shmat( )

共享存储区的附接。从逻辑上将一个共享存储区附接到进程的虚拟地址空间上。

系统调用格式：

virtaddr=shmat(shmid,addr,flag)

参数定义：

```
char *shmat(shmid,addr,flag);
int shmid,flag;
char * addr;
```

其中，shmid 是共享存储区的标识符；addr 是用户给定的，将共享存储区附接到进程的虚地址空间，当为 0 时，表示由操作系统分配一个地址；flag 其值为 SHM_RDONLY 时， 表示只能读；其值为 0 时，表示可读、可写；其值为 SHM_RND（取整）时，表示操作系统在必要时舍去这个地址。该系统调用的返回值是共享存储区所附接到的进程虚地址 viraddr。

（3）shmdt( )

把一个共享存储区从指定进程的虚地址空间断开。

系统调用格式：

```
shmdt(addr)
```

参数定义：

```
int shmdt(addr);
char addr;
```

其中，addr 是要断开连接的虚地址，亦即以前由连接的系统调用 shmat( ) 所返回的虚地址。调用成功时，返回 0 值，调用不成功，返回-1。

（4）shmctl( )

共享存储区的控制，对其状态信息进行读取和修改。

系统调用格式：

```
shmctl(shmid,cmd,buf)
```

参数定义：

```
int shmctl(shmid,cmd,buf);
int shmid, cmd;
struct shmid_ds *buf;
```

其中，buf 是用户缓冲区地址，cmd 是操作命令。命令可分为多种类型：

①用于查询有关共享存储区的情况。如其长度、当前连接的进程数、共享区的创 建者标识符等；

②用于设置或改变共享存储区的属性。如共享存储区的许可权、当前连接的进程 计数等；

③对共享存储区的加锁和解锁命令；

④删除共享存储区标识符等。

上述的查询是将 shmid 所指示的数据结构中的有关成员，放入所指示的缓冲区中；而设置是用由 buf 所指示的缓冲区内容来设置由 shmid 所指示的数据结构中的相应成员。

3.1.1 主要任务

编制一长度为 1k 的共享存储区发送和接收的程序。创建两个子进程 server 和 client 进行通信。server 端建立一个 key 为 75 的共享区，并将第一个字节置为-1，作为数据空的标志。等待其他进程发来的消息。当该字节的值发生变化时，表示收到了信息，进行处理。 然后再次把它的值设为-1，如果遇到的值为 0，则视为为结束信号，取消该队列，并退出 server。server 每接收到一次数据后显示“(server)received”。client 端建立一个 key 为 75 的共享区，当共享取得第一个字节为-1 时，server 端空闲，可发送请求。client 随即填入 9 到 0。期间等待 server 端的再次空闲。进行完这些操作后，client 退出。client 每发送一次数据后显示“(client)sent”。父进程在 server 和 client 均退出后结束。

3.1.2 具体步骤

①打开 FinalShell 窗口，创建一个 C 文件，命名为 share，并打开 share.c 文件：

```
[root@localhost ~]# touch share.c
[root@localhost ~]# vi share.c
```

②按“i”键进入 Insert 模式，在 share.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <sys/shm.h>
#include <sys/ipc.h>
```

```
#define SHMKEY 75
int shmid,i;
```

```
int *addr;
void client( )
{ int i;
   shmid=shmget(SHMKEY,1024,0777); /*打开共享存储区*/
   addr=shmat(shmid,0,0); /*获得共享存储区首地址*/
   for (i=9;i>=0;i--)
     { while (*addr!=-1);
       printf("(client) sent\n");
       *addr=i;
       }
   exit(0);
}
 void server( )
{
   shmid=shmget(SHMKEY,1024,0777|IPC_CREAT); /*创建共享存储区*/
   addr=shmat(shmid,0,0); /*获取首地址*/
   do
   {
         *addr=-1;
         while (*addr==-1);
         printf("(server) received\n");
     }while (*addr);
   shmctl(shmid,IPC_RMID,0); /*撤消共享存储区，归还资源*/
   exit(0);
}
int main( )
{
   while ((i=fork( ))==-1);

   if (!i) server( );
   system("ipcs -m");
   while ((i = fork( ))==-1);
   if (!i) client( );
   wait(0);
   wait(0);
}
```

③编译 share.c 文件，生成可执行文件 share，并执行：

```
[root@localhost ~]# gcc share.c -o share
[root@localhost ~]# ./share

------------ 共享内存段 ------------
键          shmid      拥有者      权限       字节       连接数    状态
0x0000004b  0          root        777        1024       1
(client) sent
```

(server) received
(client) sent
(server) received
(client) sent
(server) received
(client) sent
(server) received
(client) sent
(server) received
(client) sent
(server) received
(client) sent
(server) received
(client) sent
(server) received
(client) sent
(server) received

实验结果和预期一样。但在运行过程中，发现每当 client 发送一次数据后，server 要等待大约 0.1 秒才有响应。同样，之后 client 又需要等待大约 0.1 秒才发送下一个数据。出现上述应答延迟的现象是程序设计的问题。当 client 端发送了数据后，并没有任何措施通知 server 端数据已经发出，需要由 client 的查询才能感知。此时，client 端并没有放弃系统的控制权，仍然占用 CPU 的时间片。只有当系统进行调度时，切换到了 server 进程，再进行应答。这个问题，也同样存在于 server 端到 client 的应答过程中。

3.2 信号量通信模拟

信号量机制相关系统调用与接口函数：

使用与信号量机制有关的系统调用，都需包含以下头文件：

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

使用与共享存储区机制有关的接口函数说明如下：

（1）semget( )

创建、获得一个信号量集描述符。

系统调用格式：

```
semid=semget(key,nsems,flag)
```

参数定义：

```
int semget(key,nsems,flag);
key_t key;
int nsems,flag;
```

其中，key 是信号量集的名字；nsems 是指该信号量集所包含信号量的个数；flag 是用户设置的标志和访问方式，如：IPC_CREAT |0400 该信号量集是否已被创建，无则创建，是则打开；IPC_EXCL |0400 该信号量集的创建是互斥的，若已创建则失败；semid 是该系统调用返回的描述符，失败则返回-1。

（2）semop( )

对信号量集中一个或几个信号量进行操作。

系统调用格式：

```
semop(semid, sops, nsops)
```

参数定义：

```
int semop(semid, sops, nsops)
int semid
struct sembuf *sops
unsigned nsops
```

其中，semid 是上述 semget 返回的描述符；sops 是结构数组指针；nsops 表示结构数组中成员个数。struct sembuf 定义如下：

```
struct sembuf
{
        short sem_num;
        short sem_op;
        short sem_flg;
 }
```

其中，sem_num 表示所操作信号量在信号量集中位置，0 对应第一个信号量；sem_op 表示对该信号量的改变值，sem_op>0 表示进程要释放 sem_op 个共享资源，sem_op<0 表示进程要申请 sem_op 个资源；sem_flg 可取值

IPC_NOWAIT 和 SEM_UNDO，IPC_NOWAIT 表示申请资源不能满足时，进程并不等待，而是立即返回，告知申请失败，SEM_UNDO 在进程结束时，相应的操作将被取消。

系统调用的返回值：成功为 0，否则为-1。

（3）semctl( )

对信号量集的控制操作

系统调用格式：

semctl(semid, semnum, cmd, arg)

参数定义：

```
int semctl(semid, semnum, cmd, arg)
int semid, semnum, cmd;
union semun {
        semid_ds *buf;
        int *array;
        int val; } arg
```

其中，semid 是上述 semget 返回的描述符；semnum 指定对哪个信号量执行控制操作， 只对一些 cmd 操作有意义；arg 用于设置或返回信息，其类型随 cmd 操作而定；cmd 指定执行的操作，可允许的值和意义如下：

IPC_STAT 获取信号量集信息，结果在 arg.buf 中；

IPC_SET 设置信号量集信息，设置信息由 arg.buf 提供；

IPC_RMID 删除该信号量集；

SETALL 设置或更新所有信号量的值，设置值由 arg. array 提供；

GETALL 获取所有信号量的值，结果保存在 arg. array 中；

SETVAL 设置 semnum 所指定信号量的值为 arg. Val；

GETVAL 返回 semnum 所指定信号量的值；

GETNCNT 返回等待 semnum 所指定信号量的值增加的进程数，即等待该共享资源的进程数；

GETPID 返回最后一个对 semnum 所指定信号量执行 semop 操作的进程 ID；

GETZCNT 返回等待 semnum 所指定信号量的值变为 0 的进程数；

系统调用的返回值：失败为-1，成功与 cmd 相关。3.1.1 主要任务

3.2.1 主要任务

编制程序，使用信号量通信机制实现生产者—消费者问题。

3.2.2 具体步骤

①打开 FinalShell 窗口，创建一个 h 文件，命名为 semshm，并打开 semshm.h 文件：

```
[root@localhost ~]# touch semshm.h
[root@localhost ~]# vi semshm.h
```

②按“i”键进入 Insert 模式，在 semshm.h 文件中分别写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存,该头文件封装了创建信号量删除信号量，P 操作，V 操作：

```
#ifndef SEMSHM_H
#define SEMSHM_H
#define SHM_KEY 9494
#define SEM_KEY_1 9399
#define SEM_KEY_2 9595
#define BUF_SIZE 1024
#include <string.h>
union semnum
{
    int         val;
    struct semid_ds *buf;
    unsigned short    *array;
};
int sem_create(key_t key,int val)
{
    int semid;
    semid=semget(key,1,0666|IPC_CREAT);
    if(semid==-1)
    {
        perror("semget");
        exit(-1);
    }
```

```
    union semnum arg;
    arg.val=val;
    if(semctl(semid,0,SETVAL,arg)==-1)
    {
```

```
            perror("semctl");
            exit(-1);
        }
        return semid;
}
void sem_del(int semid)
{
        union semnum arg;
        arg.val=0;
        if(semctl(semid,0,IPC_RMID,arg)==-1)
        {
            perror("semctl");
            exit(-1);
        }
}
int P(int semid)
{
        struct sembuf sops={0,-1,SEM_UNDO};
        return (semop(semid,&sops,1));

}
int V(int semid)
{
        struct sembuf sops={0,+1,SEM_UNDO};
        return (semop(semid,&sops,1));
}
struct msg_data
{
        char data[BUF_SIZE];
};
void Productor(struct msg_data *msg,int i)
{
        const char * str="productid:";
        char *array[]={"1","2","3","4","5"};
        strcpy(msg->data,str);
        strcat(msg->data,array[i]);
        printf("Productor:%s\n",msg->data);
}
```

```
void Customer(struct msg_data *msg)
{
        printf("Customer:%s\n",msg->data);
}
```

```
#endif
```

③同理，创建四个 C 文件，分别命名为 allsemshm.c、sem1.c、sem2.c 和 delsemshm.c，再分别写入如下内容并保存。其中 allsemshm.c 创建了 2 个信号量用于实现生产者和消费者之间的同步问题，并创建了一个共享内存作为共享资源；sem1.c 为生产者代码；sem2.c 为消费者代码；delsemshm.c 删除所建的信号量和共享内存:

```
/* allsemshm.c */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include "semshm.h"
int main()
{
    int empty,full,shmid;
    empty=sem_create(SEM_KEY_1,1);//设置一个信号量置初值为 1
    full=sem_create(SEM_KEY_2,0); //设置一个信号量置初值为 0
    shmid=shmget(SHM_KEY,sizeof(struct msg_data),0666|IPC_CREAT);
    if(shmid==-1)
    {
        perror("shmget");
        exit(-1);
    }

    printf("empty=%d\tfull=%d\tshmid=%d\n",empty,full,shmid);
    return 0;
}
```

```
/* sem1.c*/
  #include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>
```

```
#include "semshm.h"
int main()
{
    int empty,full,shmid;
```

```
    empty=semget(SEM_KEY_1,1,0);       //获得信号量
    full=semget(SEM_KEY_2,1,0);
    shmid=shmget(SHM_KEY,0,0);         //获得共享内存
    if(empty==-1||full==-1||shmid==-1)
    {
        perror("get");
        exit(-1);
    }
    struct msg_data *buf;
    void * tmp=shmat(shmid,NULL,0);   //映射共享内存
    buf=(struct msg_data*)tmp;
    int i=0;
    for(i=0;i<5;i++)
    {
        sleep(15);
        P(empty);
int main()
{
    int empty,full,shmid;
    empty=semget(SEM_KEY_1,1,0);       //获得信号量
    full=semget(SEM_KEY_2,1,0);
    shmid=shmget(SHM_KEY,0,0);         //获得共享内存
    if(empty==-1||full==-1||shmid==-1)
    {
        perror("get");
        exit(-1);
    }
  struct msg_data *buf;
    void * tmp=shmat(shmid,NULL,0);   //映射共享内存
    buf=(struct msg_data*)tmp;
    int i=0;
    for(i=0;i<5;i++)
    {
        sleep(15);
        P(empty);
        Productor(buf,i);
        V(full);
    }
```

```
    return 0;
}
```

```
/* sem2.c */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include "semshm.h"
int main()
{
	int empty,full,shmid;
	empty=semget(SEM_KEY_1,1,0);	//获取信号量
	full=semget(SEM_KEY_2,1,0);
	shmid=shmget(SHM_KEY,0,0);	//获取共享内粗
	if(empty==-1||full==-1||shmid==-1)
	{
		perror("get");
		exit(-1);
	}

	struct msg_data *buf;
	buf=(struct msg_data*)shmat(shmid,NULL,0);	//映射共享内存地址
	int i=0;
	for(i=0;i<5;i++)
	{
		P(full);
		Customer(buf);			//消费者从共享内存取数据
		V(empty);
	}


	return 0;
}
```

```
/* delsemshm.c */
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
```

```
#include <sys/sem.h>
#include <sys/shm.h>
#include "semshm.h"
int main()
```

```
{
    int empty,full,shmid;
    empty=semget(SEM_KEY_1,1,0);
    full=semget(SEM_KEY_2,1,0);
    shmid=shmget(SHM_KEY,0,0);
    if(empty==-1||full==-1||shmid==-1)
    {
        perror("del get");
        exit(-1);
    }
    sem_del(empty);
    sem_del(full);
    struct shmid_ds *ds=(struct shmid_ds*)malloc(sizeof(struct shmid_ds));
    shmctl(shmid,IPC_RMID,ds);
    printf("del success !\n");
    return 0;
}
```

④编译四个 C 文件，生成可执行文件 allsemshm、sem1、sem2 和 delsemshm：

```
[root@localhost ~]# gcc allsemshm.c -o allsemshm
[root@localhost ~]# gcc sem1.c -o sem1
[root@localhost ~]# gcc sem2.c -o sem2
[root@localhost ~]# gcc delsemshm.c -o delsemshm
```

⑤先运行 allsemshm，建立信号量和共享内存，再运行 senm1 生产者程序，在另一个终端运行 sem2 消费者程序，观察程序的输出，再调用 delsemshm 程序删除信号量和共享内存：

```
[root@localhost ~]# ./allsemshm
empty=229376    full=262145    shmid=163840
[root@localhost ~]# ./sem1
Productor:productid:1
Productor:productid:2
Productor:productid:3
Productor:productid:4
Productor:productid:5
[root@localhost ~]# ./delsemshm
[root@localhost ~]# ./sem2
Customer:productid:1
Customer:productid:2
```

```
Customer:productid:3
Customer:productid:4
Customer:productid:5
```

首先创建了 empty 和 full 两个信号量用来实现进程同步问题，并创建了一个

共享内存做为共享资源，接着生产者会生产 5 次，只有缓冲区内容被取走时才能进行生产操作，同时开启了 1 个消费者程序。只有缓冲区有内容时，消费者才能执行他的动作。

四、实验讨论

1、比较信号量和共享存储区这两种通信机制中数据传输的性能。

2、当多个进程都使用 shmget 函数创建共享存储区的时候，key 值是否可以相同？如果 key 值出现相同的情况，会出现什么问题？如何解决？

3、信号量通信模拟实验中，sem1 是生产者，sem2 是消费者，如果改为 2 个生产者或 2 个消费者，情况会是如何？请修改程序完成实验。